Instructor Run Sheet - Project Lab Level 2: The Night Watchman

Document: Instructor Run Sheet - Project Lab Level 2: The Night Watchman
Version: v1.0.0
Author: Celaya Solutions
Contact: hello@celayasolutions.com
Date: 2026-09-06
SHA256: cd17b3b2777e43fab51ce091f1c937b16afa1766ad7461d8f68826ee78adbf55
Chain: n/a
Tx: [not anchored]

Outcome

Specify, run, and test a scheduled page watcher that records each round, alerts only on a meaningful change, and stops through a tested switch.

Alignment

Target	Practice	Evidence
Define bounded autonomy	Six-line spec before enabling	Own page, exact signal/rule, trigger, issue, never list
Explain persistence	Baseline and unchanged rounds	Last good state read on a fresh runner
Judge a useful alert	OPEN to PAUSED, then repeat	One issue with both values; no duplicate
Diagnose failure	Trace a failed receipt to its stage	Failure is not called no change
Stop work	Disable, false variable, cancel, refresh	Disabled state and no active/queued work
Deliver proof	Private Markdown upload and reopen	Correct file and submission receipt

Teaching stance

This is a small automation, not a model-driven decision maker. Do not imply that a language model is necessary to read two exact statuses. The point is authority, state, evidence, and the off switch. Explain new terms when first used; demonstrate one UI action, then let learners perform it.

The 90 minutes starts after preparation. Each live learner uses their own suitable Windows/Mac computer and their existing fork. Partner/paper/saved routes assess understanding while clearly leaving individual live-launch evidence open.

Prepare before learners arrive

1. Read preparation, lesson, project map, and Level 2 verification. Distribute an exact candidate commit that contains the watcher; the older Level 1 tag does not.

2. Use a designated public practice fork. Confirm your role, default branch, Issues, workflow installation, branch policy, and the off switch. Do not use the canonical repository or a client repository.

3. Follow the learner setup literally on Windows and Mac. Run `uv run --frozen zta test watchman` and the Level 1 regression check. A hosted CI test is not a full device pilot.

4. In that approved practice fork, verify fresh baseline, unchanged, changed, repeated unchanged, issue body, stored state, failure receipt, and off switch. Record run links and times in the verification file. Disable after testing. Do not call local fixtures hosted results.

5. Make the worksheet, saved packet, manual card, answer key, slides, and PDFs available offline. Keep predictions covered before showing saved actuals.

6. Sign in as a test student, open Level 2, upload and reopen a sample evidence file, and record the actual controls privately. Preserve existing students and submissions. If access is unavailable, record that gate and plan a later receipt check.

7. Recheck the linked GitHub controls and schedule facts before teaching. Announce the candidate's unperformed gates rather than improvising a pass.

Safety boundary

Only the supplied public page in the learner's fork. Temporary GitHub token only, limited to contents/issues write for this job. Those scopes are repository-wide; the code confines writes to its state file and practice issues. No write tokens for pull requests, no pasted personal token, no private pages or people tracking, and no automatic purchase, reply, deletion, external posting, or spending.

Public output is only synthetic status and project receipts. Learner notes stay in ignored `.zta/`. Disabling new triggers is separate from cancelling already queued/active runs. Leave every practice workflow disabled and its enable variable false at the end.

90-minute schedule

Block	Minutes
Ready check and show the stop control	7
Explain look, compare, tell with saved state	8
Demonstrate receipts and one failure	8
Task 1: spec, peer prediction, baseline prediction	12
Task 2: enable, baseline, unchanged	16
Task 2: OPEN to PAUSED and issue check	13
Repeat check and log review	6
Task 3: disable, cancel, refresh, record	10
Private submission, reopen, and exit	10
Total	**90**

Do not consume stop/submission time waiting for Actions. After two attempts or five minutes on a block, use the saved route and record what still needs a live check. Keep prepared installation outside these blocks.

Facilitation plan

Open with an observable promise

Show the practice page and ask: "Would a date in the footer be worth waking someone?" Point to `watch-value`. Ask learners to explain why the first run has no old value. Show `.watch-state/watchman.json`, an issue, a failed receipt, and the disable control before enabling your demonstration.

Keep predictions ahead of results

Ask learners to save a prediction and time before each Run workflow click. For each completed round, have them point to previous, current, decision, time, and the source commit. A green badge does not replace this explanation. A gray skipped job did not check the page.

Use a fresh manual run after editing the page. An old job rerun retains its old source commit; it can create a confusing result even if the current page has changed. Wait for one run to complete before starting another.

Explain recovery with a simple sequence

The watcher saves a pending alert before sending. If sending may have succeeded, it searches for the same issue marker; it does not blindly send again. If an issue exists but the final state write failed, a later run completes that pending transition. Only after that can a new observation be compared. The advanced mechanics are in the project README; learners need the practical rule: preserve receipts and ask for help instead of deleting state.

Coach the switch honestly

Learners disable the workflow, change the variable to false, cancel outstanding work, and refresh. They record the visible disabled state and unavailable manual control. Do not claim a scheduled job failed to appear at a future time unless that time was actually observed. Waiting a minute proves little because GitHub scheduling can be delayed.

Coach code ownership and submission

A code commit belongs to the learner's fork. A submission receipt belongs to the private course. A saved packet demonstrates evaluation, not the learner's own hosted system. Use the same rubric, while keeping those different claims explicit.

Passing proof

A complete watchman spec; baseline, unchanged, and changed runs; one useful alert and log trail; a never list; and proof that the learner used the off switch.

Grading guide

Check	Meets	Return for revision when
Spec	All six lines predict the supplied behavior	No exact signal, rule, address, or permitted issue
Predictions	Written and timed before each run	Expectations copied after seeing results
Main runs	Three different decisions with dated receipts	Failed/skipped is called unchanged or baseline
Alert and repeat	Exactly one issue for OPEN to PAUSED; repeat creates none	Only email shown, values absent, or duplicate issue
State	Learner explains why the last good value survives failure	Learner deletes state to hide an unexplained result
Switch	Disabled, false variable, no queued/active run, refreshed control checked	Merely names the switch or closes laptop
Evidence route	Own live / local / partner / saved explicitly labeled	Authored logs treated as live launch
Submission	Correct private Markdown file reopened	Only export, GitHub link, or screenshot supplied

Learner success: six completed spec lines; predictions before all four checks; three main verdicts and one repeat verdict; issue with both values; three dated main receipts; switch observations; six exit answers; private receipt or a named access block. Do not grade machine or queue speed. Application release success additionally requires the unperformed device/hosted/schedule/submission pilots to be completed and recorded.

Access and support

Use the manual card for either assistant or no assistant. The local rehearsal needs no model/API and makes no network request. A learner can explain local before/after results without buying coding-assistant access. Keep that route distinct from GitHub proof.

Allow partner reading, paper predictions, extra time for keyboard navigation, and Spanish anchors. Supply the complete English PDF and the saved packet offline. Full Spanish learner translation remains a separate course phase.

Fallbacks

- Account/policy outage: use the saved packet after the stop rule; record the missing own-fork pilot.

- Page failed validation: keep old state, restore the one known marker, run anew.

- State write denied: inspect branch policy; do not remove protections from an employer repository.

- Definite issue rejection: fix the specific permission/Issues block and start one new run.

- Uncertain issue delivery: disable and inspect state plus all relevant open/closed issues. Follow the project README's reconciliation procedure; never simply toggle `attempted` to false.

- Missing artifact because setup never ran: preserve the GitHub job failure log.

- Course platform unavailable: retain `PROJECT-LAB-02.md` privately and submit later. No public-issue workaround.

After class

Record each learner's route, runtime and setup time, first failure category, live checks still owed, and submission status. Verify every instructor-owned watcher is disabled, its variable is false, and no run remains queued or active. Save useful non-sensitive receipts before artifacts expire. Do not start the Railway extension or Level 3 during this handoff.